

Technical Description of the Project Architecture, Data Pipeline, and Training Workflow

ml2_final_project

May 30, 2026

Abstract

This document explains the project at the level a new reader needs first: how the repository is organized, how the data moves from raw corpora to processed tensors, how the training code is split into reusable modules, and how the experiment folders are produced.

Contents

1	Proposal	3
2	Repository Overview	3
2.1	High-Level Directory Tree	3
2.2	What Each Major Folder Means	4
3	End-to-End Data Flow	4
3.1	Data Pipeline Diagram	5
4	How Data Should Be Organized	5
4.1	Raw Data Layout	5
4.2	Processed Data Layout	5
4.3	What Goes Into a .pt File	6
4.4	Feature Extraction Parameters	6
4.5	Filtering Rules and Rationale	6
5	Preprocessing Step by Step	7
5.1	Typical Workflow	7
5.2	Example Preprocessing Command	7
5.3	Why This Stage Matters	7
6	Dataset Loading and Batching	8
6.1	Dataset Classes	8
6.2	Default Training Mix	8
6.3	What Each Sample Returns	8
6.4	Batching Strategy	8
7	System Architecture: Train Structure	9
7.1	Pipeline Diagram	9
7.2	Vocoder Diagram	9

8	Loss Functions	10
9	Inference Workflow	10
9.1	Inference Architecture Diagram	10
10	Training Code Organization in src	11
10.1	Module Map	11
10.2	Why This Modularization Helps	11
10.3	Recommended Experiment Flow	12
11	How to Run Experiments	12
11.1	Experiment Directory Tree	12
11.2	What Each Output Means	12
11.3	Example Commands	12
11.4	Monitoring a Run	13
12	Step-by-Step Guide from Download to Training	13
12.1	1. Download the datasets	13
12.2	2. Verify the folder structure	13
12.3	3. Run preprocessing	13
12.4	4. Inspect a few outputs	13
12.5	5. Confirm the dataset loader can read the manifests	13
12.6	6. Start with a small experiment	14
12.7	7. Scale up	14
12.8	8. Review the experiment artifacts	14
13	How to Modularize the Project Cleanly	14
13.1	Adding a New Dataset	14
13.2	Adding a New Model Component	14
13.3	Adding a New Experiment Variant	14
13.4	Adding Better Debugging	14
14	Practical Example of the Whole Workflow	14
15	Summary	15

1 Proposal

The goal of this project is to build a model for cross-ligual voice conversion. The model will be trained with the datasets: LibriSpeech-PT, TTS Portuguese Corpus, LibriSpeech-EN.

2 Repository Overview

The repository is divided into a few clear responsibilities: raw data storage, preprocessing, reusable code, experiment outputs, and documentation.

2.1 High-Level Directory Tree

```
ml2_final_project/
|-- data/
|   |-- raw/
|   |   |-- libriSpeech-pt/
|   |   |-- tts-portuguese-Corpora/
|   |-- processed/
|   |   |-- libriSpeech-pt/
|   |   |   |-- train/
|   |   |   |   |-- libriSpeech_mels_metadata.csv
|   |   |   |-- test/
|   |   |   |   |-- libriSpeech_mels_metadata.csv
|   |   |-- tts-portuguese/
|   |   |   |-- mels_metadata.csv
|   |-- interim/
|   |-- external/
|-- scripts/
|   |-- download_datasets/
|   |-- preprocess/
|   |   |-- preprocess_libriSpeech-pt.py
|   |   |-- preprocess_mels_tts_portuguese.py
|-- src/
|   |-- data/
|   |   |-- first_step_data_loaders/
|   |   |-- datasets.py
|   |-- models/
|   |   |-- AcousticDecoder.py
|   |   |-- GST.py
|   |   |-- TextEncoder.py
|   |   |-- HiFi-GAN.py
|   |-- training/
|   |   |-- train_first_step/
|   |   |   |-- train.py
|   |   |   |-- model_loader.py
|   |   |   |-- losses.py
|   |   |   |-- train_utils.py
|   |   |   |-- text_processing.py
|   |   |   |-- configs.py
|   |   |   |-- run_training.py
|   |   |-- checkpoint_utils.py
|-- experiments/
```

```
|  '-- step_1/
|      '-- attempt_<timestamp>/
|-- docs/
```

2.2 What Each Major Folder Means

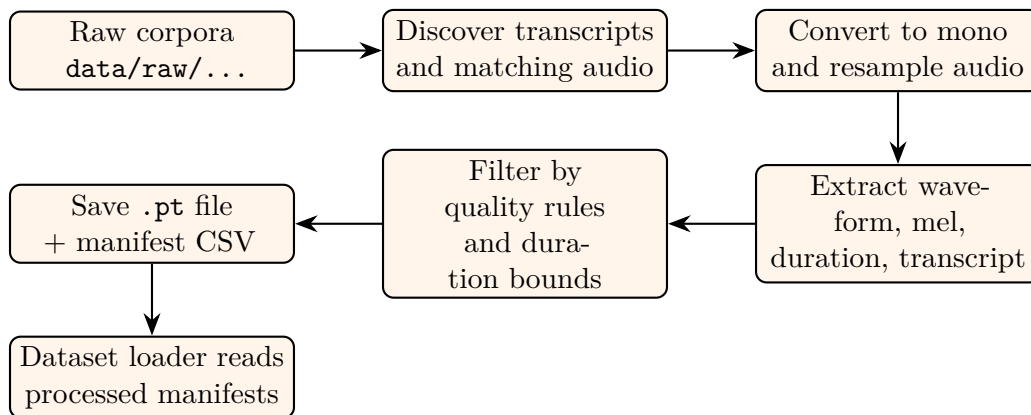
Folder	Role in the project
<code>data/raw</code>	Original corpora exactly as downloaded. Nothing here should depend on a model version or a feature extraction recipe.
<code>data/processed</code>	Processed artifacts such as <code>.pt</code> files and metadata CSV manifests used by the dataset layer.
<code>data/interim</code>	Temporary intermediate outputs or scratch files that do not belong in the final processed dataset.
<code>scripts/preprocess</code>	Reproducible preprocessing scripts that convert raw audio into filtered, normalized, indexed training samples.
<code>src/data</code>	Dataset wrappers and batching helpers that turn manifests into PyTorch objects.
<code>src/models</code>	Reusable neural network components. The architecture text in this document does not depend on these files as its source of truth.
<code>src/training</code>	Training orchestration, losses, configuration presets, checkpointing, and experiment utilities.
<code>experiments</code>	Timestamped or named experiment outputs. Each run keeps its config, checkpoints, logs, and TensorBoard data together.
<code>docs</code>	Technical descriptions, reports, and the LaTeX sources for the project documentation.

3 End-to-End Data Flow

The project follows a simple lifecycle:

1. Download raw corpora into `data/raw`.
2. Preprocess audio and transcripts into standardized tensors.
3. Save per-utterance artifacts and metadata manifests in `data/processed`.
4. Load the processed manifests with the dataset classes in `src/data`.
5. Build the training experiment from the modules in `src/training`.
6. Save checkpoints, logs, and plots under `experiments/step_1`.

3.1 Data Pipeline Diagram



4 How Data Should Be Organized

The project treats raw data and processed data as two different products. Raw data is the source of truth; processed data is a reusable training cache.

4.1 Raw Data Layout

Place each corpus in its own directory under `data/raw`. A practical layout is:

```
data/raw/  
|-- libriSpeech-pt/  
|   '-- mls_portuguese_opus/  
|       |-- train/  
|       |-- dev/  
|       '-- test/  
'-- tts-portuguese-Corpora/  
    '-- TTS-Portuguese-Corpus/  
        |-- audio/  
        |-- text/  
        '-- metadata/
```

The exact nested folder names may differ depending on how the corpus was downloaded, but the rule is the same: raw content stays grouped by dataset and is not overwritten by preprocessing.

4.2 Processed Data Layout

The processed area should mirror the dataset identity and the split that was used during preprocessing. The current project expects these main outputs:

```
data/processed/  
|-- libriSpeech-pt/  
|   |-- train/  
|   |   |-- <utt_id>.pt  
|   |   '-- librispeech_mels_metadata.csv  
|   '-- test/  
|       |-- <utt_id>.pt  
|       '-- librispeech_mels_metadata.csv  
'-- tts_portuguese/
```

```
|-- <utt_id>.pt  
'-- mels_metadata.csv
```

The dataset classes in `src/data/first_step_data_loaders/datasets.py` use these manifests directly. A sample CSV row can look like this:

```
utt_id,text,duration,mel_path,audio_path  
pt_000123,ola mundo,2.83,data/processed/libriSpeech-pt/train/pt_000123.pt,data/raw/  
libriSpeech-pt/.../pt_000123.flac
```

4.3 What Goes Into a .pt File

Each serialized sample should keep everything needed to train or inspect the example later:

- **waveform**: the resampled mono waveform tensor.
- **mel**: the log-mel spectrogram tensor, with 80 mel bins.
- **sr**: the sample rate used by preprocessing, usually 22050 Hz.
- **duration**: utterance duration in seconds.
- **text**: the transcript text after normalization.
- **utt_id**: the unique utterance identifier.
- **audio_path**: the original audio path relative to the corpus root when available.

4.4 Feature Extraction Parameters

The preprocessing recipe is fixed so that every sample is compatible with the model input shape:

- Sample rate: 22050 Hz.
- Mel bins: 80.
- FFT size: 1024.
- Window length: 1024.
- Hop length: 256.
- Mel frequency range: 0 Hz to 8000 Hz.

4.5 Filtering Rules and Rationale

A practical preprocessing pipeline should reject examples that are too short, too long, or malformed. In this project, the intended rule set is:

- Minimum duration: 1.0 second.
- Maximum duration: 20.0 seconds.
- Mono waveform only.

- Resampled to the target sample rate.
- Transcript and audio must be aligned and non-empty.

These bounds keep padding under control, reduce memory spikes, and make batch shapes more predictable.

5 Preprocessing Step by Step

The preprocessing scripts in `scripts/preprocess` are responsible for building the manifests used by training.

5.1 Typical Workflow

1. Read the raw corpus root.
2. Find transcript files and enumerate utterances.
3. Match each transcript line to the corresponding audio file.
4. Load audio with `torchaudio`.
5. Convert multichannel audio to mono if necessary.
6. Resample to 22050 Hz.
7. Compute the mel spectrogram with the fixed STFT configuration.
8. Compute duration and apply the duration filter.
9. Save the tensor payload to one `.pt` file per utterance.
10. Write the metadata CSV manifest.
11. Optionally generate plots or sanity checks for a sample subset.

5.2 Example Preprocessing Command

```
python scripts/preprocess/preprocess_libriSpeech-pt.py \
  --input-dir data/raw/libriSpeech-pt/mls_portuguese_opus \
  --out-dir data/processed/libriSpeech-pt/train \
  --target-sr 22050 \
  --n-mels 80 \
  --n-fft 1024 \
  --hop-length 256 \
  --win-length 1024 \
  --min-duration 1.0 \
  --max-duration 10.0
```

5.3 Why This Stage Matters

This step is where the project becomes reproducible. If preprocessing is deterministic and documented, the same raw corpus always produces the same processed artifacts, and later training runs can be compared fairly.

6 Dataset Loading and Batching

The module `src/data/first_step_data_loaders/datasets.py` turns processed manifests into PyTorch datasets.

6.1 Dataset Classes

The current design uses three useful building blocks:

- `LibriSpeechPTDataset`: loads one LibriSpeech-PT split, typically `train` or `test`.
- `TTSPortugueseDataset`: loads the TTS Portuguese processed manifest.
- `CombinedFirstStepDataset`: concatenates multiple datasets into one training set.

6.2 Default Training Mix

The repository memory and dataset helpers agree on the default composition: LibriSpeech-PT train, LibriSpeech-PT test, and TTS Portuguese are combined for first-step training. That means the training code can treat the three sources as a single pool of processed examples.

6.3 What Each Sample Returns

Each dataset item returns a dictionary with the core information needed by the training loop:

- `mel`
- `waveform`
- `sr`
- `duration`
- `text`
- `utt_id`
- `mel_path`
- `audio_path`
- `source`

6.4 Batching Strategy

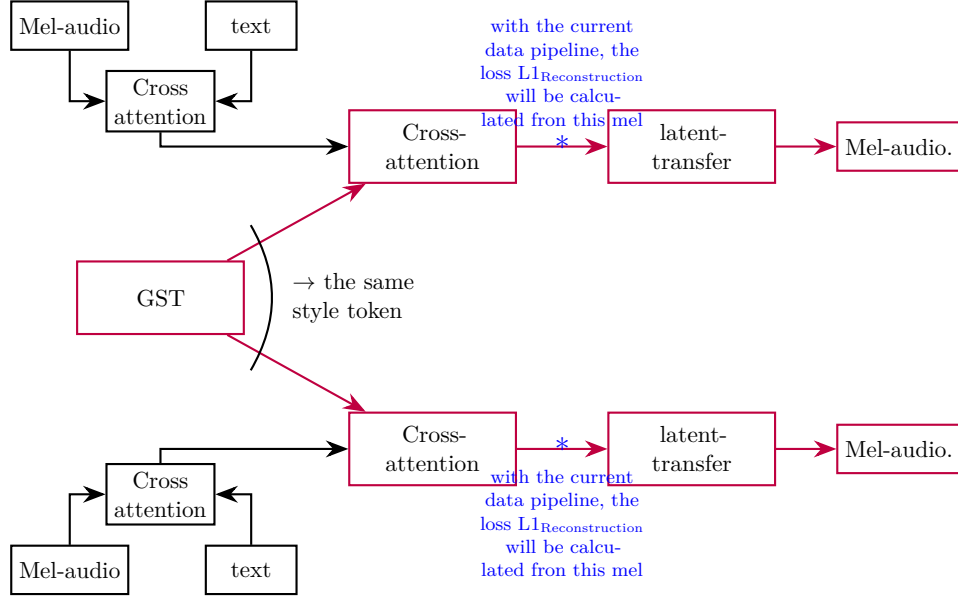
Mel spectrograms and waveforms have variable lengths, so the collate function pads them along the time axis to the longest sample in the batch. The batch also keeps metadata such as the text string, utterance ID, duration, and source corpus name.

This is important for two reasons:

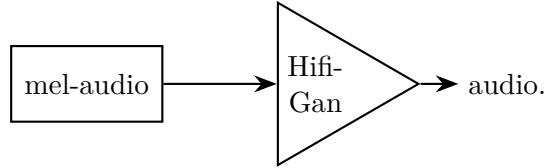
1. Neural networks expect tensors with consistent batch dimensions.
2. Metadata is needed for debugging and for later experiment analysis.

7 System Architecture: Train Structure

The training phase is designed as a dual-pathway architecture processing both textual and acoustic features. The network maps Mel-spectrogram audio and corresponding text through cross-attention mechanisms, while simultaneously extracting stylistic features via a shared Global Style Token (GST) module.



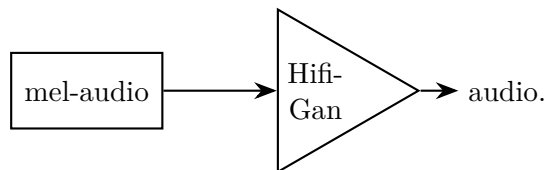
7.1 Pipeline Diagram



During training, the central **GST** module processes Mel-audio to output "the same style token," which is injected into both the top and bottom large **Cross-attention** modules. The outputs pass through a **latent-transfer** module before yielding the predicted **Mel-audio**. Note that with the current data pipeline, the $L1$ Reconstruction loss will be explicitly calculated from the mel extracted at the marked (*) junction prior to latent-transfer.

7.2 Vocoder Diagram

For converting Mel-spectrograms back into waveform audio, a HiFi-GAN vocoder is utilized:



8 Loss Functions

The loss objective combines multiple weighted criteria to govern perceptual quality, reconstruction accuracy across both languages, stylistic diversity, and emotional consistency. The complete formula is defined strictly as:

$$\begin{aligned} \text{Loss} = & w_1 \text{Loss}_{\text{perceptual}}(\cdot) \\ & + w_2 L1_{\text{Reconstruction}}(\text{predicted-Mel-english}, \text{target}) \\ & + w_2 L1_{\text{Reconstruction}}(\text{predicted-Mel-Portuguese}, \text{target}) \\ & + w_3 L_{\text{style-diversity}}(\text{all-the-style-tokens}) \\ & + w_4 L_{\text{emotional-compatibility}}(\dots) \end{aligned}$$

Implementation Notes:

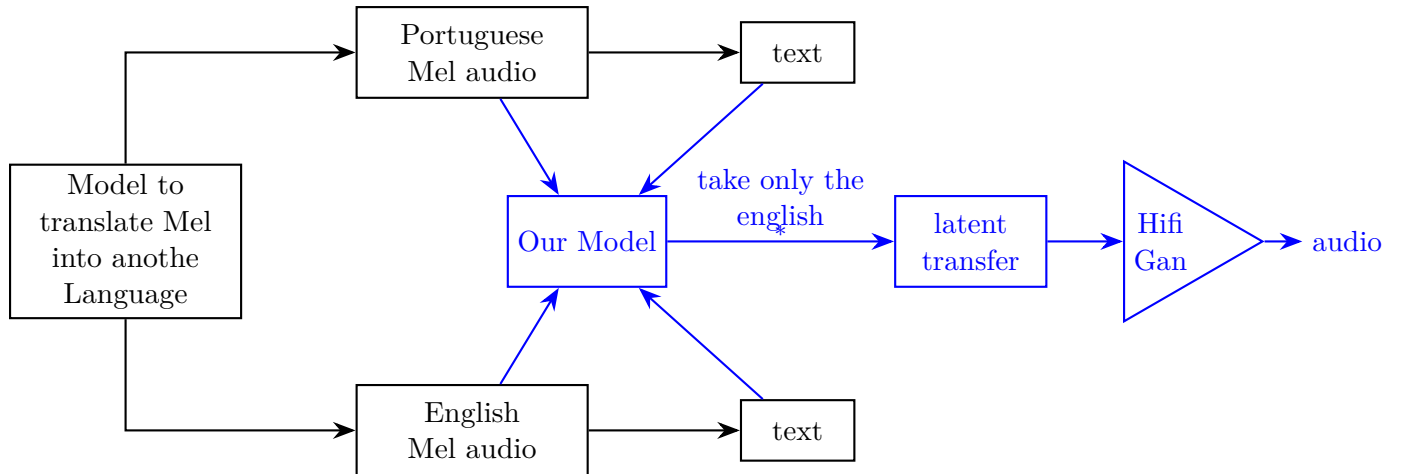
- $\text{Loss}_{\text{perceptual}}$: Evaluated using the emotion classification network.
- $L_{\text{style-diversity}}$: Calculated based on the cosine similarity of all style tokens.
- $L_{\text{emotional-compatibility}}$: Currently undefined, but intended to utilize the classification network to assist in enforcing emotional consistency.
- Note that both reconstruction losses (English and Portuguese) share the same weight scalar w_2 in the standard formulation.

9 Inference Workflow

The inference stage describes how a translated language (English) is synthesized using audio from a source language (Portuguese). The network processes dual-language Mel audio and text, isolating the translated English features for output.

"For inference we will take a portuguese audio and output the english one. Our model is trained take: portuguese and english audio. So,"

9.1 Inference Architecture Diagram



** Maybe as a next step we can bail our own*

The final model must receive only the Portuguese audio and output the English audio. For the first time, we will train only the model presented in the diagram architecture. The models for text-to-speech and speech-to-text are pre-trained.

After having a good style token, we will try to build a model for text-to-speech and speech-to-text.

10 Training Code Organization in `src`

The training pipeline is split into small modules so that each concern has one main place in the codebase.

10.1 Module Map

Module	What it does
<code>train.py</code>	Main training entry point. Parses arguments, builds datasets, instantiates the model, runs training and validation, and writes outputs.
<code>model_loader.py</code>	Creates the full first-step TTS model from the submodules in <code>src/models</code> .
<code>losses.py</code>	Implements the objective terms used by training.
<code>train_utils.py</code>	Contains the epoch loops, checkpoint helpers, TensorBoard logging, and validation utilities.
<code>text_processing.py</code>	Converts transcript strings into token sequences.
<code>configs.py</code>	Stores named training presets such as quick test, balanced, production, and lightweight runs.
<code>run_training.py</code>	Convenience launcher that selects a preset and forwards its arguments to <code>train.py</code> .
<code>checkpoint_utils.py</code>	Utility commands for listing, comparing, and inspecting saved checkpoints.
<code>test_setup.py</code>	Sanity-check script for verifying that the environment and model loading work correctly.

10.2 Why This Modularization Helps

Splitting the code this way makes the project easier to maintain and easier to review.

- If a preprocessing bug appears, you only edit the scripts under `scripts/preprocess`.
- If the model plan changes, you update the architecture description in this document and the corresponding training code together.
- If the loss definition changes, you touch `losses.py` without rewriting the training loop.
- If you want a new experiment setup, you add a preset in `configs.py` instead of duplicating a command by hand.

10.3 Recommended Experiment Flow

The practical order for a run is:

1. Verify the environment with the setup test.
2. Preprocess the raw corpora into `data/processed`.
3. Inspect one or two manifest rows and one or two `.pt` files.
4. Launch a small preset such as `quick_test`.
5. Check the experiment directory and TensorBoard logs.
6. Scale up to `balanced` or `production` after the pipeline is stable.

11 How to Run Experiments

Experiments are stored in `experiments/step_1`. Every run creates a separate directory, usually named with a timestamp or a custom experiment name.

11.1 Experiment Directory Tree

```
experiments/  
  '-- step_1/  
    '-- attempt_YYYYMMDD_HHMMSS/  
      |-- config.json  
      |-- checkpoints/  
        |-- epoch_0001.pt  
        |-- epoch_0002.pt  
        '-- best.pt  
      |-- tensorboard/  
      '-- logs/
```

11.2 What Each Output Means

- `config.json`: the exact hyperparameters used for the run.
- `checkpoints/`: saved training states that allow resuming or evaluating later.
- `tensorboard/`: event files for loss curves, metrics, and qualitative logs.
- `logs/`: human-readable training logs or auxiliary text output.

11.3 Example Commands

```
# Quick smoke test  
python src/training/train_first_step/run_training.py quick_test  
  
# Balanced default run  
python src/training/train_first_step/run_training.py balanced
```

```
# Full custom training
python src/training/train_first_step/train.py \
  --num-epochs 100 \
  --batch-size 32 \
  --learning-rate 1e-3 \
  --weight-reconstruction 1.0 \
  --weight-diversity 0.5 \
  --acoustic-decoder-hidden-size 256 \
  --acoustic-decoder-num-layers 3 \
  --style-embedding-dim 128 \
  --num-workers 4 \
  --experiment-name cross_lingual_training
```

11.4 Monitoring a Run

You can monitor training with TensorBoard:

```
tensorboard --logdir experiments/step_1/<experiment_name>/tensorboard
```

Useful curves include total loss, reconstruction loss, diversity loss, validation metrics, and any audio examples logged from the vocoder.

12 Step-by-Step Guide from Download to Training

This section is the shortest path from a fresh corpus to a reproducible model run.

12.1 1. Download the datasets

Place the raw archives in `data/raw`. Do not preprocess before the download tree is complete.

12.2 2. Verify the folder structure

Make sure the corpus layout is readable by the preprocessing scripts. If a dataset uses different folder names, adapt the input path before running the script.

12.3 3. Run preprocessing

Use the scripts in `scripts/preprocess` to create the processed dataset. The result should be one `.pt` file per utterance plus a manifest CSV.

12.4 4. Inspect a few outputs

Open one manifest row and one saved sample to confirm that the mel tensor, waveform, duration, and transcript look sane.

12.5 5. Confirm the dataset loader can read the manifests

The dataset module should successfully resolve the `mel_path` entries and return dictionaries with consistent keys.

12.6 6. Start with a small experiment

Run `quick_test` first. It is designed for a fast sanity check, not for final quality.

12.7 7. Scale up

After the small run works, move to `balanced` or a custom training command.

12.8 8. Review the experiment artifacts

Check `config.json`, the best checkpoint, the TensorBoard curves, and any generated audio logs.

13 How to Modularize the Project Cleanly

If you want to extend the project without creating a monolithic script, keep each responsibility in the same layer where it already belongs.

13.1 Adding a New Dataset

Add a new preprocessing script under `scripts/preprocess`, write the processed files under `data/processed/<new_dataset>`, and create a matching dataset wrapper under `src/data/first_step_data_loaders`.

13.2 Adding a New Model Component

Create the neural module in `src/models`, then wire it into the implementation layer that uses the conceptual architecture described above. If the component changes the loss behavior, update `losses.py` at the same time.

13.3 Adding a New Experiment Variant

Put the hyperparameters in `configs.py`. That keeps command-line invocations short and makes experiments easy to reproduce.

13.4 Adding Better Debugging

Log examples, shapes, and checkpoints from `train_utils.py`. Keep debug output next to the training loop, not scattered across unrelated modules.

14 Practical Example of the Whole Workflow

A complete minimal run might look like this:

1. Download LibriSpeech-PT and TTS Portuguese into `data/raw`.
2. Preprocess both corpora and write manifests into `data/processed`.
3. Launch `python src/training/train_first_step/run_training.py quick_test`.
4. Confirm that checkpoints are created under `experiments/step_1`.

5. Open TensorBoard and check that losses decrease as expected.
6. If everything is stable, rerun with the balanced preset.

15 Summary

The project is organized around a simple engineering rule: raw data stays in **data/raw**, processed artifacts stay in **data/processed**, reusable code lives in **src**, and experiment outputs live in **experiments**. The model architecture in this document is diagram-driven, not code-driven. That keeps the explanation aligned with the conceptual design the project is trying to communicate.